Homework 1

Gabriela Rubio

January 26, 2026

1. instructions per second $= 1742 \times 10^{15}$
   time in seconds $= \frac{(\text{total instructions})}{(1742 \times 10^{15})}$
   time in minutes $= \frac{(\text{total instructions})}{(60 \times 1742 \times 10^{15})}$
   time in hours $= \frac{(\text{total instructions})}{(60 \times 60 \times 1742 \times 10^{15})}$
   time in days $= \frac{(\text{total instructions})}{(24 \times 60 \times 60 \times 1742 \times 10^{15})}$
   time in years $= \frac{(\text{total instructions})}{(365 \times 24 \times 60 \times 60 \times 1742 \times 10^{15})}$
   time in centuries $= \frac{(\text{total instructions})}{(100 \times 365 \times 24 \times 60 \times 60 \times 1742 \times 10^{15})}$

   (a) $n = 100$
   total instructions $= 2^n = 2^{100}$
   time in centuries $= \frac{2^{100}}{(100 \times 365 \times 24 \times 60 \times 60 \times 1742 \times 10^{15})}$
   230.75164662073178 centuries

   (b) $n = 1000$
   total instructions $= 2^n = 2^{1000}$
   time in centuries $= \frac{2^{1000}}{(100 \times 365 \times 24 \times 60 \times 60 \times 1742 \times 10^{15})}$
   1.9504773273645153e+273 centuries

2.

2^{500}	$\log(\log n)^2$	$\log_4 n$	$\log n$	$\log^3 n$	$\sqrt{n}$
$2^{\log n}$	$n \log n$	$n^2 \log^5 n$	n^3	2^n	$n!$

3. (a) $f(n) = \Theta(g(n))$
   (b) $f(n) = O(g(n))$
   (c) $f(n) = \Omega(g(n))$
   (d) $f(n) = O(g(n))$
   (e) $f(n) = \Omega(g(n))$
   (f) $f(n) = \Theta(g(n))$

4. (a) **Algorithm description:** The algorithm iterates through the items once. For each item, if adding it would exceed K, we check if it can be a solution on its own and return it. Otherwise, we add the item to the subset, and check if our current subset is a solution.

   (b) **Pseudocode:**

```
Algorithm: ApproximateKnapsack
Input: real number K, array items[1...n]
Output: array where the total is >= K/2 and <= K

array ApproximateKnapsack(K, items) {
    total = 0;
    subset = {};

    for each item in items {
       if total + item > K {
            if item >= K/2 and item <= K {
```

```
                return {item};
            }
        }

        else {
            add item to subset;
            total = total + item;
            if total >= K/2 {
                return subset;
            }
        }
    }
}
```

(c) **Correctness:** The algorithm is correct because there are only two possible cases for an output: returning a subset or returning a single item. In the first case, we return a subset only after checking that the total is at least K/2 but no more than K, since we only add elements to the subset if it will make the total smaller than K, and we return right away after reaching K/2. On the second case, we return a single number only after checking that the number is at least K/2 but no more than K, we don't need to check for anything else, so we know we have a solution. The problem states that there's always a solution, so we are guaranteed to find something.

(d) **Time Analysis:**

```
array ApproximateKnapsack(K, items) {
    total = 0;                                  // O(1)
    subset = {};                                // O(1)

    for each item in items {                    // T(n)
       if total + item > K {                    // O(1)
            if item >= K/2 and item <= K {      // O(1)
                return {item};                  // O(1)
            }
        }

        else {
            add item to subset;                 // O(1)
            total = total + item;               // O(1)
            if total >= K/2 {                   // O(1)
                return subset;                  // O(1)
            }
        }
    }
}
```

The algorithm runs in **O(n)** time. The algorithm performs 2 constant-time operations for initialization (lines 1-2). The loop executes at most n times, and each iteration performs at most 7 constant-time operations (comparisons, arithmetic, assignments, and returns). Therefore, $T(n) = 2 + 7n = O(n)$.